

Assignment #7: MQTT Clients and Brokers

1. Major MQTT Client Libraries

- Eclipse Paho (Java, Python, C, JavaScript)
- MQTT.js (JavaScript/Node.js)
- Mosquitto Client Libraries (C)
- HiveMQ MQTT Client (Java)
- gmqtt (Python)
- Async MQTT Client (Python asyncio)
- MQTTnet (.NET/C#)
- umqtt (MicroPython)
- WolfMQTT (Embedded C)

2. Major MQTT Broker Implementations

- Eclipse Mosquitto
- HiveMQ
- EMQX
- RabbitMQ (with MQTT plugin)
- VerneMQ
- ActiveMQ
- AWS IoT Core (Managed MQTT Broker)
- Azure IoT Hub (MQTT support)

3. Feature Comparison of MQTT Brokers

Eclipse Mosquitto:

- Lightweight and open-source
- Suitable for small to medium deployments
- Supports MQTT 3.1 and 5.0
- Limited scalability features

HiveMQ:

- Enterprise-grade broker
- High scalability and clustering
- Advanced security and monitoring
- MQTT 5.0 support

EMQX:

- Highly scalable and distributed
- Supports millions of connections
- Built-in dashboard and rule engine
- Strong cloud-native features

RabbitMQ:

- Multi-protocol messaging broker
- MQTT support via plugin
- Good for integration with existing systems
- Not specialized for MQTT

VerneMQ:

- Distributed and scalable
- High availability and fault tolerance
- Good for IoT use cases

ActiveMQ:

- Supports multiple protocols including MQTT
- Enterprise integration focus
- Moderate scalability

AWS IoT Core:

- Fully managed cloud service
- Highly scalable and secure
- Integrates with AWS ecosystem

Azure IoT Hub:

- Managed service with MQTT support
- Strong device management features
- Integrates with Azure services